



Roadmap to a Functional App

A summer-scale plan to take **quad** from a visual prototype with mock data to a real app where Notre Dame students can find gigs, join hangouts, and message one another.

Where we are today

Layer	Status
Visual design	Done — splash, welcome, gigs feed, map, hangouts, messages inbox, chat threads
Routing	Done — expo-router with stack + tabs
Brand	Done — logo, NamePlaque, grey/white palette
Auth	Not started
Database	Not started — Supabase client wired but no project yet
Real-time	Not started
Backend logic	None — everything is mock data
Native build	None — runs only as web/Expo Go
Distribution	Not started

The app **looks** real. Nothing is real yet.

Phase 1 — Foundation

Week 1–2 · the unsexy plumbing. Without it, no other phase is possible.

Supabase project setup

- ☐ Create a Supabase project at supabase.com (free tier)

- ☐ Copy the project URL and anon key into `.env`
- ☐ Confirm the existing `lib/supabase.ts` client connects (write a tiny test query)
- ☐ Configure Auth → Email Auth → restrict signup to email domains containing `nd.edu`

Database schema

Design and migrate the core tables. Each should have RLS (row-level security) on from day one.

- ☐ `profiles` — id (FK to auth.users), display_name, initials, year, major, dorm, avatar_url, bio, verified_at
- ☐ `gigs` — id, poster_id, title, description, category, payout_cents, location_label, lat, lon, posted_at, status, accepted_by, deadline_at
- ☐ `hangouts` — id, host_id, title, vibe, location_label, lat, lon, starts_at, max_people, description
- ☐ `hangout_attendees` — hangout_id, user_id, joined_at
- ☐ `conversations` — id, gig_id (nullable), hangout_id (nullable), created_at
- ☐ `conversation_members` — conversation_id, user_id, last_read_at
- ☐ `messages` — id, conversation_id, sender_id, body, sent_at
- ☐ `reports` — id, reporter_id, target_user_id, target_kind, target_id, reason, created_at
- ☐ Write RLS policies: users can only read/write their own data; gigs and hangouts are publicly readable to verified students; messages only readable by conversation members

Auth flows

- ☐ **Sign up** screen with `.edu` email + password
- ☐ Email verification — link sent on signup, opens app to confirm screen
- ☐ **Sign in** screen
- ☐ **Forgot password** flow
- ☐ **Sign out** action in settings
- ☐ Session persistence (verify it survives app close)
- ☐ Auth-gate the tabs: redirect to `/welcome` → `/sign-in` if no session

Profile creation

- ☐ First-time profile setup screen after signup (name, year, major, optional dorm, optional photo)
- ☐ **Profile** tab — your own profile + edit
- ☐ View other users' profiles (read-only)
- ☐ Storage bucket `avatars` with policy: users can write their own avatar only

Phase 1 done when: a new student can sign up with their `.edu` email, verify it, set up a profile, sign out, sign back in, and see their profile data persisted.

Phase 2 — Gigs end-to-end

Week 3–4 · the wedge. Get this fully working before touching hangouts.

Posting

- ☐ Wire the **Post a gig** FAB on the Gigs tab to a real form
- ☐ Form fields: title, description, category, payout, location, optional photo, optional deadline
- ☐ Submit → insert into `gigs` table with `status = 'open'`
- ☐ Show "your post is live" confirmation, navigate back to feed
- ☐ Validation: required fields, payout > \$0, basic word filter

Reading the feed

- ☐ Replace the mock `MOCK_GIGS` array with a real Supabase query
- ☐ Filter by category from the chips (server-side)
- ☐ Pagination — load 20 at a time, fetch more on scroll
- ☐ Realtime subscription on the `gigs` table → new posts appear live
- ☐ Sort: newest first, with a toggle for "closest"

Gig detail screen

- ☐ Tap a gig card → navigate to `/gig/[id]`
- ☐ Full description, photo, poster's profile snippet, payout, location, posted-time
- ☐ **"I'll do it"** button → creates a conversation, navigates to chat
- ☐ If you're the poster: see list of applicants, "Accept" button
- ☐ On accept: `status = 'accepted'`, hide gig from public feed
- ☐ On completion: poster taps "Mark complete" → triggers review prompt

Reviews (lightweight)

- ☐ After a gig is done, both parties leave a 1–5 star rating + 1-sentence review
- ☐ Aggregate rating shown on user profiles ("4.8 ★ · 12 gigs")
- ☐ Hide partial ratings until a user has 3+ reviews

Phase 2 done when: a student can post a gig, another can apply, they can chat to coordinate, mark it complete, and rate each other — without writing any code.

Phase 3 — Hangouts + real messaging

Week 5–6 · the social layer.

Hangouts

- ☐ Replace mock `MOCK_HANGOUTS` with `hangouts` table query
- ☐ **Start a hangout** FAB → form (title, vibe, location, when, max_people, description)
- ☐ **I'm in** button → insert into `hangout_attendees`
- ☐ Hangout detail screen with attendee list, description, location pin
- ☐ Auto-create a group conversation linked to the hangout
- ☐ Leave hangout button (removes from attendees + group chat)
- ☐ Host can cancel hangout (sends a system message to the group)

Messaging (the real version)

- ☐ Replace mock `CONVOS` with a real query joining conversations, members, latest messages
- ☐ Show only conversations where you're a member (RLS handles this)
- ☐ Inbox sorted by latest message timestamp
- ☐ Unread count: messages where `sent_at > my last_read_at`

Chat thread (the real version)

- ☐ Replace mock conversation lookup with a real query of `messages`
- ☐ **Realtime subscription** so incoming messages append live
- ☐ Send button: insert into `messages`, optimistically append, clear input
- ☐ Update `last_read_at` when the user opens the thread
- ☐ Show the gig/hangout context line as a real link to the source

Optional but nice

- ☐ Typing indicator (Supabase presence)
- ☐ Read receipts
- ☐ Image attachments — upload to Supabase Storage

Phase 3 done when: two real students on two real phones can RSVP to the same hangout and see

their messages appear live in the group chat.

Phase 4 — Map + native build

Week 7 · escape the web prototype.

Real map data

- ☐ Replace static `PINS` with a query of all open gigs and upcoming hangouts
- ☐ Pin emoji chosen by category
- ☐ Tap pin → popup with title and a "View" link to the detail screen
- ☐ Filter chips on the map (Gigs / Hangouts / both)
- ☐ "Center on me" button (requires location permission)

Native map (replace the iframe)

- ☐ Install `react-native-maps`
- ☐ Render `MapView` on native, keep iframe on web
- ☐ Custom markers using emoji as the marker label
- ☐ Build a development client (`eas build --profile development`) — Expo Go can't load native modules

Location capture

- ☐ When posting, allow tapping a map to set lat/lon (instead of typing a label)
- ☐ Reverse-geocode the tapped point to a campus landmark name (use a hardcoded ND landmarks list for v1)

Phase 4 done when: the Map tab shows live data from your database with proper pins on iOS, Android, and web.

Phase 5 — Trust, safety, polish

Week 8 part A.

Reporting & blocking

- ☐ Long-press / "..." menu on any gig, hangout, message, or profile → "Report"

- ☐ Insert into `reports` , optional reason text
- ☐ Block user — adds to a `blocks` table; blocked users' content is filtered out everywhere
- ☐ You get an email per new report; manual review for v1

Notifications

- ☐ Set up Expo Notifications, store push tokens on `profiles`
- ☐ Trigger via Supabase Edge Functions on: new application, accepted application, new message, hangout join
- ☐ In-app notification feed (last 50 events for you)

Settings screen

- ☐ Sign out
- ☐ Edit profile
- ☐ Notification preferences (toggle each event type)
- ☐ Privacy: "show me on the map" toggle
- ☐ Delete account (cascades correctly)

Onboarding & permissions

- ☐ First-launch tutorial overlay (3 quick screens)
- ☐ Permission prompts at the right time (notifications when entering Messages, location when opening Map)

Phase 6 — Launch prep

Week 8 part B.

Legal & store listings

- ☐ Privacy policy (Termly boilerplate, then customize for student-data specifics)
- ☐ Terms of service
- ☐ Apple Developer account (\$99/yr)
- ☐ Google Play Console account (\$25 one-time)
- ☐ App Store screenshots (5–8 per device size)
- ☐ App description, keywords, category
- ☐ Production app icon (1024×1024)

- ☐ Production splash screen

Builds & beta

- ☐ `eas build --profile production` for iOS and Android
- ☐ TestFlight build distributed to ~20 ND students for closed beta
- ☐ Iterate on feedback for 1–2 weeks
- ☐ Submit to App Store and Play Store

Seeding & launch

- ☐ Pre-seed 30+ gigs and hangouts before launch day
- ☐ Notre Dame–specific marketing: dorm flyers, campus subreddit, GroupMe drops, freshman orientation table
- ☐ Track day-1, day-7, day-30 retention from launch

Explicitly out of scope for v1

Feature	Why not yet
In-app payments	Users settle via Venmo / cash. Stripe Connect is a 3-month project on its own.
Second campus	Wedge depends on density. Don't expand until ND has a strong base.
Background checks	Premature for a campus-only app with <code>.edu</code> verification.
Group hangouts > 50 people	YAGNI. Cap at 20 for v1.
AI matchmaking / recommendations	Build the manual feed first. Personalization is a post-PMF problem.
Web dashboard for admins	Use the Supabase dashboard directly.

Critical risks to watch

1. **Cold start on day 1** — if the feed is empty when the first 50 students open the app, they bounce and don't come back. Pre-seeding is non-negotiable.
2. **.edu verification false positives** — anyone with an `nd.edu` email includes alumni and faculty. Decide whether that's a feature or a bug before launch.
3. **Liability** — students meeting up via your app to do gigs (especially "rides to the airport") creates real liability. A clear ToS disclaimer and an in-app safety guide are mandatory before launch.
4. **Trust collapse on a single bad actor** — one scam or one harassment incident can poison the well on a small campus. Reporting + blocking must work on day 1, not day 30.

What to do this week

1. Create the Supabase project
2. Build the schema with RLS
3. Wire up sign-up and sign-in screens against real auth

4. Replace the mock gigs feed with real database queries
5. Stop building new visual screens until users can actually log in.